

ClimateOS

Technical Stack & Implementation Architecture

AI-Powered Climate Disaster Intelligence & Response Platform

Source: ClimateOS — Hackathon PRD v2.0, 7 pages.

Purpose: Complete technical-stack mapping for every PRD section, from product loop and AI engines to maps, routing, offline mode, simulation, security, deployment and future expansion.

Important: The PRD defines architecture and requirements, but does not name specific frameworks or vendors. Technologies below are therefore proposed implementation choices, not claims about what is already specified in the PRD.

Core loop: Predict → Sense → Verify → Understand → Prioritize → Act → Learn

1. Executive Technical Stack

Layer	Proposed stack	Role
Citizen Web	Next.js + React + TypeScript + Tailwind	Personal risk, alerts, reports, shelters, routes
Citizen Mobile	Expo + React Native (optional)	Camera, GPS, notifications, offline support
Government	Next.js + React + MapLibre GL JS	Command center and live risk map
API	Python + FastAPI + Pydantic	Authentication, orchestration and typed APIs
Application services	FastAPI modules + SQLAlchemy	Reports, incidents, shelters, routes, resources
AI	Python + scikit-learn + PyTorch/Transformers	Risk, verification, priority and recommendations
Database	PostgreSQL + PostGIS	Relational + geospatial data
Cache/events	Redis + WebSockets/SSE	Live updates, caching and simulation events
Maps/routing	MapLibre + OSM + OSRM/GraphHopper	2D/2.5D map and route calculation
Files	S3-compatible object storage	Citizen photos/voice evidence
DevOps	Docker + GitHub Actions	Reproducible builds and deployment

The PRD's lightweight architecture is: Citizen Web/Mobile → Government Dashboard → API Layer → Application Services → AI Services → Data Layer → Digital Twin. [filecite turn0file0 L175-L183](#)

2. Section-by-Section Stack

2.1 Product Vision & Core Loop

- Next.js/React for user interfaces; FastAPI for orchestration; Redis + WebSockets/SSE for event updates; PostgreSQL for durable state; Python AI services for the intelligence loop.
- Implement each transition as a traceable event: data → analysis → verification → risk → recommendation → human action → citizen feedback → recalculation.

2.2 Data Sources

- Python ingestion using httpx/requests; pandas or Polars for CSV/Parquet; GeoJSON/GeoPackage/OpenStreetMap data into PostGIS; simulated sensors and synthetic citizen reports generated by Python scenario scripts.
- MVP sources in the PRD include weather/API data, historical disaster data, maps/geospatial data, population/infrastructure datasets, simulated sensors and synthetic citizen reports. [filecite turn0file0 L184-L188](#)

2.3 AI Fusion Core

- Python is the AI layer. Use scikit-learn for reliable baseline models, PyTorch/Transformers only where deep learning materially helps, and Pydantic for strict model input/output schemas.
- Return confidence and supporting metadata with every important AI result so the government UI can explain it.

2.4 Risk Prediction Engine

- Features: rainfall/weather, historical patterns, geography, population/infrastructure and sensor signals.
- Start with an interpretable weighted score or gradient-boosting model; output risk_score, category, affected_zone, severity and confidence. Store snapshots in PostgreSQL/PostGIS.

2.5 AI Report Verification

- Inputs: citizen text, photo, GPS, timestamp, nearby reports, weather/risk and sensor data.
- Use NLP/image models only as needed; combine model confidence with spatial/time consistency checks. Output high-confidence / needs-review / low-confidence status.

2.6 Impact & Priority Engine

- Python scoring service. Rank incidents using people affected, injuries, trapped status, severity, accessibility, infrastructure impact and current zone risk.

- Persist component scores so the ranking is explainable and reproducible.

2.7 Recommendation Engine

- FastAPI service consumes verified intelligence and produces action, target, reason, confidence and human-review status.
- Use deterministic safety constraints in the MVP. Government authorities accept/reject recommendations; the PRD explicitly excludes fully autonomous emergency decisions. [\[filecite\]turn0file0\[L195-L205\]](#)

2.8 Citizen Ground-Truth Network

- Next.js web or Expo mobile; browser/device geolocation; camera/photo upload; optional voice input; FastAPI submission endpoint; S3-compatible evidence storage; PostgreSQL/PostGIS report records.
- The PRD allows location, photo, description, optional voice input, category and severity in a report. [\[filecite\]turn0file0\[L78-L90\]](#)

2.9 Dynamic Risk Map & Digital Twin

- MapLibre GL JS + GeoJSON frontend; PostgreSQL/PostGIS backend; layers for hazard zones, incidents, verified incidents, roads, blocked roads, hospitals, shelters, resources and population-risk areas.
- Keep the Digital Twin 2D/2.5D for MVP, matching the PRD's explicit direction. [\[filecite\]turn0file0\[L91-L95\]](#)

2.10 Personalized Risk & Action

- PostGIS spatial queries map user location to risk zones. Combine current risk, shelter availability, road state and user preferences. Deliver actions via web UI and optional push notifications.

2.11 Dynamic Safe-Route Engine

- OSRM or GraphHopper provides base routes. A ClimateOS risk layer adds flood probability, hazard severity, blocked roads, incidents, accessibility and emergency-access constraints.
- Store road blockage/hazard geometry in PostGIS and recalculate whenever new ground truth invalidates a route.

2.12 Rescue & Resource Prioritization

- PostgreSQL/PostGIS for resources and locations; Python ranking service; Redis cache for the current queue; government UI for deployment recommendations and human approval.

2.13 Government Command Center

- Next.js + TypeScript + MapLibre. Live updates through WebSockets/SSE. Core views: city situation, active zones, critical incidents, AI recommendations, prioritized feed, resources and live map.
- The PRD says the government interface should prioritize actions rather than raw data. [\[filecite\]turn0file0\[L111-L120\]](#)

2.14 Citizen Application

- Responsive Next.js experience for MVP; Expo/React Native can add native camera, notifications and stronger offline support.
- Home screen surfaces personal risk, active alerts, recommended action, reporting and shelter discovery. [\[filecite\]turn0file0\[L121-L123\]](#)

2.15 Shelter Intelligence

- PostGIS stores shelter location; PostgreSQL stores capacity/occupancy/accessibility/status. Suitability query combines distance, availability, accessibility and hazard safety; routing engine returns the safest route.

2.16 Explainable AI

- Use stored rule components for deterministic scores and SHAP/feature contribution methods for compatible ML models.
- Expose confidence and major factors such as rainfall, water level, terrain and verified reports. [\[filecite\]turn0file0\[L127-L129\]](#)

2.17 Offline / Low Connectivity

- Web: service worker + IndexedDB. Mobile: SQLite/AsyncStorage if implemented. Cache last-known risk, emergency instructions, shelter data, emergency contacts and relevant map data.
- Queue reports locally and retry when connectivity returns. SMS fallback remains future architecture, consistent with the PRD. [filecite\turn0file0\L130-L132](#)

2.18 Disaster Simulation Mode

- Python simulation service with JSON/YAML scenarios. Publish synthetic rainfall, water-level, incident and road events through Redis/WebSockets. Dashboard visualizes state changes in real time.
- Simulation is a key hackathon feature in the PRD and should demonstrate the complete adaptive loop. [filecite\turn0file0\L133-L135](#)

2.19 Signature Hackathon Demo

- Seed Zone A, rainfall, water level, roads, shelters, citizens and resources. Trigger prediction → risk rise → citizen reports → AI verification → map update → recommendations → personalized route → new flooded-route report → recalculation → learning.
- This exactly follows the PRD's 10-step signature demo. [filecite\turn0file0\L138-L149](#)

2.20 Functional Requirements

- FR-01/02: Python risk engine + PostGIS + MapLibre. FR-03: report UI + FastAPI + PostGIS. FR-04/05: AI verification + priority service. FR-06: notification service. FR-07: OSRM/GraphHopper + risk scoring. FR-08: shelter service. FR-09: resource ranking. FR-10: government dashboard. FR-11: Redis + WebSockets/SSE. FR-12: explainability metadata. FR-13: Python simulation. FR-14: event-driven feedback and recalculation.

2.21 Non-Functional Requirements

- Performance: lightweight clients, server-side AI, Redis caching. Scalability: stateless APIs + PostgreSQL/PostGIS + containers. Reliability: durable writes, retries and offline cache. Security: TLS, RBAC, secrets management, encrypted storage. Explainability: confidence/factors. Accessibility: semantic UI and short emergency actions.

2.22 Lightweight Technical Architecture

- Citizen Web/Mobile → Next.js/React or Expo. Government Dashboard → Next.js/React + MapLibre. API → FastAPI. Application Services → modular FastAPI services. AI → Python. Data → PostgreSQL/PostGIS + Redis + object storage. Digital Twin → PostGIS-backed geospatial state rendered by MapLibre.

2.23 Data Sources

- MVP: weather/API, historical disaster data, geographic/map data, population/infrastructure datasets, simulated sensors and synthetic reports. Future: IoT sensors, river gauges, government systems, satellite feeds, emergency services and traffic systems. [filecite\turn0file0\L184-L188](#)

2.24 MVP Scope

- Citizen: location, risk, alerts, reports, safe route and shelters. Government: risk map, incident feed, verification, priority, resources and simulation. AI: risk scoring, classification, confidence, impact, ranking and recommendations. Digital Twin: lightweight 2D/2.5D representation. [filecite\turn0file0\L189-L194](#)

2.25 Explicitly Out of MVP

- Full IoT deployment, production government integrations, complex 3D, drones, AR/VR, blockchain, large-scale social monitoring, autonomous emergency decisions, advanced satellite processing and nationwide deployment are future/out-of-scope.

2.26 Development Priorities

- P0: risk engine, reporting, AI verification, dynamic map, prioritization. P1: personalized alerts, routes, shelter intelligence, resources, government recommendations. P2: offline, SMS architecture, explainable AI, simulation. P3: live IoT, satellite, government integrations, multi-disaster expansion. [filecite\turn0file0\L206-L215](#)

2.27 Success Metrics

- AI: risk/classification accuracy, verification confidence, ranking consistency. Response: report-to-authority, verification-to-recommendation and route recalculation times. Citizen: submission time, alert comprehension, successful route/shelter recommendation. System: dashboard/API response time and simultaneous simulated

incidents. [\[filecite\turn0file0\L216-L221\]](#)

2.28 Key Differentiator

- Technically, ClimateOS is a closed adaptive intelligence loop rather than a standalone weather app, alert system, GIS dashboard or prediction model. The architecture therefore emphasizes state updates and feedback.

2.29 Winning Pitch

- The demo should visually connect live map changes, AI verification, ranked incidents, recommendations, citizen personalization and route adaptation. Use deterministic seeded data for reliability.

2.30 Product North Star

- Optimize the end-to-end latency from 'Something is happening' to 'The right person is taking the right action.' Measure the complete pipeline, not just isolated model accuracy. [\[filecite\turn0file0\L233-L235\]](#)

2.31 Final Product Definition

- Predict → risk engine. Sense → citizen/environmental data. Verify → AI verification. Understand → Digital Twin + risk map. Prioritize → ranking. Act → recommendations/routes/resources. Learn → feedback-driven recalculation. This matches the PRD's final definition. [\[filecite\turn0file0\L236-L245\]](#)

3. Core Data Model

Entity	Important fields	Technology
User	id, role, location, preferences	PostgreSQL
Incident	type, severity, geometry, description, status, confidence	PostgreSQL + PostGIS
CitizenReport	reporter, GPS, photo, voice, timestamp, category	PostgreSQL + object storage
Verification	status, confidence, factors, model_version	PostgreSQL
RiskZone	geometry, score, category, severity, confidence, valid_at	PostGIS
RiskSnapshot	zone, score, factors, model_version, timestamp	PostgreSQL
RoadSegment	geometry, blocked/status, hazard_score	PostGIS
Shelter	geometry, capacity, occupancy, accessibility, status	PostGIS
Resource	type, quantity, location, availability	PostGIS
Recommendation	action, target, reason, confidence, decision	PostgreSQL
Route	origin, destination, geometry, risk_score, distance, status	PostGIS + Redis cache
SimulationEvent	scenario, type, payload, timestamp	PostgreSQL + Redis

4. API Surface

Method	Endpoint	Purpose
POST	/auth/login	Authenticate
GET	/risk/current?lat=&lng=	Personalized current risk
GET	/risk/zones	Risk-map GeoJSON
POST	/reports	Citizen report
GET	/incidents	Incident feed
POST	/incidents/{id}/verify	Review/verify
GET	/shelters/nearby	Suitable shelters
POST	/routes/safe	Risk-aware route
GET	/resources	Resource status
GET	/priorities	Ranked incidents/resources
POST	/recommendations/{id}/decision	Authority decision
POST	/simulation/start	Start scenario
POST	/simulation/events	Inject event
GET	/stream	WebSocket/SSE live stream

5. Suggested Repository Structure

```
climateos/  
├─ apps/  
│   ├── citizen-web/ # Next.js citizen app  
│   ├── government-dashboard/ # Next.js command center  
│   └─ mobile/ # Optional Expo app  
├─ services/  
│   ├── api/ # FastAPI API layer  
│   ├── risk-engine/ # Risk prediction  
│   └─ verification/ # Report verification
```

- | └─ priority-engine/ # Impact + ranking
- | └─ recommendation/ # Human-reviewable actions
- | └─ routing/ # Safe-route scoring
- | └─ simulation/ # Disaster simulation
- └─ packages/
 - | └─ schemas/ # Shared contracts
 - | └─ ui/ # Shared UI
- └─ data/
 - | └─ seed/
 - | └─ scenarios/
- └─ infra/
 - | └─ docker/
 - | └─ migrations/
- └─ docs/

6. Security, Privacy & Safety

- Authentication: OAuth 2.0 / OpenID Connect.
- Authorization: backend RBAC for citizen, responder and authority roles.
- Transport: HTTPS/TLS everywhere.
- Secrets: environment variables locally; managed secret store in deployment.
- Database: least-privilege accounts, encrypted backups and migrations.
- Location privacy: expose exact citizen locations only where operationally necessary.
- Uploads: file-type/size validation, signed object-storage URLs and malware scanning where available.
- Auditability: log AI recommendation, human decision, timestamp and actor.
- AI safety: confidence thresholds and human review; never treat recommendations as autonomous emergency commands.

7. Deployment & DevOps

Area	Recommended implementation
Local development	Docker Compose
CI/CD	GitHub Actions
Frontend	Managed Next.js hosting such as Vercel or equivalent
Backend	Container platform such as ECS/Fargate, Render, Railway or equivalent
Database	Managed PostgreSQL + PostGIS
Cache	Managed Redis
Object storage	S3-compatible bucket
Observability	OpenTelemetry + centralized logs; Sentry optional
Testing	Pytest + Playwright + frontend unit/component tests
API docs	FastAPI-generated OpenAPI
Infrastructure	Docker-first; Terraform optional after MVP

8. Hackathon Build Order

- Phase 1 — Foundation: PostgreSQL/PostGIS, FastAPI, Next.js, auth, base map and seed city.
- Phase 2 — P0: risk scoring, citizen reports, verification, priority ranking and map layers.
- Phase 3 — P1: shelters, safe routes, recommendations, authority review and personalized actions.

- Phase 4 — Live loop: Redis, WebSockets/SSE and simulation.
- Phase 5 — Demo polish: explainability cards, command-center UX, deterministic scenario and metrics.
- Phase 6 — P2 only if stable: offline cache, SMS architecture and deeper resilience.

9. Final One-Page Stack

Category	Final choice
Frontend	Next.js + React + TypeScript + Tailwind CSS
Mobile	Expo + React Native (optional)
Maps	MapLibre GL JS + OpenStreetMap-compatible data
Backend	Python + FastAPI + Pydantic
AI/ML	Python + scikit-learn + PyTorch/Transformers where needed
Data processing	pandas/Polars + NumPy
Database	PostgreSQL + PostGIS
Cache/events	Redis + WebSockets/SSE
Routing	OSRM or GraphHopper + custom risk scoring
Storage	S3-compatible object storage
Auth	OAuth 2.0 / OIDC + RBAC
DevOps	Docker + GitHub Actions
Observability	OpenTelemetry + centralized logging
Testing	Pytest + Playwright + frontend tests

Source note: The uploaded PRD explicitly states that heavy AI processing should run server-side, the system should support multiple zones/reports/disaster types, critical information should survive degraded connectivity, and security must protect location/personal/government data. [filecite\turn0file0\L165-L171](#)